How a bouquet upload is saved

Student file: Acuesta_John_Constantine_Finals_LabActivity1_HOW_IT_WORKS.pdf
Theme: Petal Express (Laravel 11 + Blade + Breeze, SQLite).

Part 1 is the reading, in plain words, including aspect ratio.
Part 2 is one figure per page. A code figure shows the file
name in the title bar and the real line numbers beside it.
A page figure is the website at 1280 by 900.

You upload a JPG, a PNG, or a WebP. Intervention writes one
watermarked WebP. Blade prints that URL in an img tag.
There is no separate thumbnail file.

Notes: docs/FINALS-LAB-ACT-1-CODE.md
Demo login: admin@test.com / 123
Show the watermark on /bouquets/{id}, not on /posts/{id}.

1. What this lab adds

Labs 2 and 3 already save a bouquet row, check the admin role, validate the form, and soft-delete the row. This lab adds the photo. The visitor may send a JPG, a PNG, or a WebP. The app does not keep that file.

Intervention Image reads the upload, shrinks it if it is wider than 1000 pixels, stamps public/images/watermark.png on the bottom-right, and encodes a new WebP. Laravel stores the path in showcase_flowers.image. Blade later prints that path inside an img tag. The browser downloads the file. Intervention does not run again on the way out.

2. Words you need

A route is the URL. POST /admin/posts is the Add bouquet form. A controller is the PHP class for that URL. Ours is AdminPostController. A form request checks the fields first. Ours is PostRequest. A model is the class for one table. Post uses the table showcase_flowers, not a table named posts.

A migration created the table. The photo column is a string named image. It holds a path, not pixels. Blade is the HTML file. A disk is Laravel's name for a folder. The public disk root is storage/app/public, set in config/filesystems.php.

3. The two files, and why the names differ

The stamp is petal-express/public/images/watermark.png. The name stays watermark.png because saveWatermarkedImage() always opens that exact path. It is part of the project. You do not upload it per bouquet. On this machine it is a PNG, 560 by 160, with a transparent background.

The bouquet photo is storage/app/public/products/ plus a UUID plus .webp. products/ keeps uploads in one folder. Str::uuid() makes a random 8-4-4-4-12 id so two files named rose.jpg cannot overwrite each other. The file is written before Post::create() knows the new row id, so the name is not the id. .webp matches Format::WEBP. The original filename is discarded.

A real row on this machine is bouquet 101, Progress Report Bouquet. The column stores products/8bfef734-0640-407f-af5d-a6f47d24a581.webp. Pillow reports WebP, 800 by 1150. Width stayed 800 because the upload was narrower than 1000. Height stayed 1150, so the shape stayed tall.

4. How the browser is allowed to see storage

The web server may read the public folder. It may not read storage/app/public by itself. php artisan storage:link creates public/storage as a shortcut to storage/app/public. The browser URL is /storage/ plus the column value, with no drive letter and no storage/app prefix.

For bouquet 101 the tag src is /storage/products/8bfef734-0640-407f-af5d-a6f47d24a581.webp. displayImageUrl() returns a path starting with /storage/. The browser follows public/storage to the file.

5. What happens when you click Save bouquet

create.blade.php posts to the route admin.posts.store. enctype="multipart/form-data" is required. Without it, PHP never receives the file. The input name is image. @csrf is Laravel's check that the form came from our site.

routes/web.php runs auth and role:admin before the controller. A guest is sent to login. A member gets 403. Intervention does not run for them.

PostRequest then checks image. On create the rules are required, image, mimes jpg jpeg png webp, and max 3072. max is kilobytes, so the cap is 3 MB. On edit, required becomes nullable. A blank file input keeps the old photo. If a rule fails, Laravel returns to the form and saveWatermarkedImage() never starts.

store() does not resize the photo itself. It calls saveWatermarkedImage($request->file('image')) and puts the returned string in the image column, next to title, body, price, category_id, and auth()->id(). The redirect flash is Bouquet added. Watermark applied.

6. What Intervention does, line by line

Image::decode($file) reads the bytes. After that line the code does not care that the original was JPG or PNG. This machine's PHP has the gd extension, so Intervention uses GD. The controller does not name the driver.

scaleDown(width: 1000) shrinks the photo only when it is wider than 1000 pixels. Height changes by the same factor. A tall photo stays tall. A square stays square. favorite ceo is 1000 by 1000 because that upload was a square wider than 1000.

public_path('images/watermark.png') opens the stamp. If the file is missing, the method throws Watermark not found before Post::create(), so you get no row and no WebP. The mark is scaled to 38 percent of the photo width and never wider than 320 pixels.

insert($watermark, 12, 12, Alignment::BOTTOM_RIGHT, 0.55) paints it. The 12s are the gap from the corner. BOTTOM_RIGHT picks the corner. 0.55 keeps 55 percent of the mark's opacity. In the library the argument is named transparency, but the GD driver treats 1 as full strength.

The path is products/ plus Str::uuid() plus .webp. encodeUsingFormat(Format::WEBP, quality: 80) makes the bytes. Storage::disk('public')->put() writes them under storage/app/public. The method returns the path. That return value is the only thing the database stores. PHP deletes the temp upload at the end of the request.

6a. Why the mark is words inside a PNG

The stamp looks like the words petal express. The file is still an image, public/images/watermark.png, a PNG that is 560 by 160 with a transparent background. saveWatermarkedImage() never draws letters. It calls Image::decode on that PNG, then insert().

Intervention can draw text with a text() method. This lab does not use it. text() needs a font file on the computer that runs the upload. If that font is missing in the demo, every save fails. The PNG already has the font, the blue color, and the letter spacing, so the controller only has to place the file.

A picture of flowers would cover the bouquet. Words with a transparent background stay readable in the corner and leave the flowers visible. One PNG is shared by every upload. The controller always opens the same path.

6c. What else Intervention does

The watermark is one step. The same method uses Intervention for the rest of the photo. Image::decode reads the upload, so JPG, PNG, and WebP become one picture. scaleDown(width: 1000) resizes the photo and keeps the aspect ratio. scaleDown runs again on the stamp. insert() paints the stamp. encodeUsingFormat(Format::WEBP, quality: 80) makes the WebP bytes.

Alignment::BOTTOM_RIGHT and Format::WEBP are settings for those calls. They are not extra effects. This lab does not blur, crop, flip, or build a second thumbnail file. CSS shrinks the one WebP on the page.

6b. Aspect ratio, in plain words

Aspect ratio is how wide a picture is compared with how tall it is. A square is 1 to 1. The stamp is 560 by 160, which is 3.5 to 1. Bouquet 101 is 800 by 1150, about 16 to 23.

The admin list box is h-12 and w-12. That is 48 by 48 CSS pixels, a square. The class object-cover fills the square and cuts off the extra height. For an 800 by 1150 file, the square keeps the middle and cuts about 175 pixels from the top and 175 from the bottom. The mark sits in that bottom strip, so the small list photo can hide it.

The preview class is object-contain. That fits the whole photo in the box and leaves empty space on the sides. On a 1280 by 900 window the preview box measured 846 by 675, and the file inside it is still 800 by 1150. The mark shows there.

The public detail class is object-cover with a minimum height of 420 pixels. On that same window the box measured 414 by 595. That matches 800 to 1150, so this screen shows the whole photo, including the mark. A different window width can cut it, because the class is still object-cover.

7. The lines that display the image

Saving the file does not show it. An img tag shows it. The src must be a URL. One method builds that URL for every screen that should show the upload.

```
Post.php lines 57-66
if ($this->image) {
    if (Storage::disk('public')->exists($this->image)) {
        return '/storage/'.ltrim($this->image, '/');
    }
}
return $this->galleryImageUrl();
```

Line 59 skips empty columns. Line 61 checks the file is really on the public disk. Line 62 is the display line for an upload. For bouquet 101 it returns /storage/products/8bfef734-0640-407f-af5d-a6f47d24a581.webp. Storage::url() would prefix APP_URL. A demo at 127.0.0.1:8000 would then request localhost and the image would break. The path that starts with /storage/ uses the host already in the address bar.

galleryImageUrl() at line 70 runs only when there is no upload. It cycles public/images/bouquets/01.jpg through 06.jpg by row id. Those JPGs are not watermarked. Seeded rows use them.

The controller only loads the row. PageController::bouquetsShow lines 57 to 61 call findOrFail($id) and return view('pages.bouquets-show'). The method displayImageUrl() runs later, inside the Blade file, when the img tag is printed.

```
bouquets-show.blade.php line 5
<img src="{{ $bouquet->displayImageUrl() }}" alt="{{ $bouquet->title }}">
```

The double braces print the string. The browser requests that src. The shortcut public/storage serves the WebP. Intervention is not called on this request.

The public list does the same call one step earlier. bouquets-index.blade.php line 47, home.blade.php line 55, and member/favorites.blade.php line 19 pass :image="$bouquet->displayImageUrl()" into the showcase-list-item component. The tag is showcase-list-item.blade.php lines 13 to 16, src="{{ $image }}", with classes h-20 w-20. Those classes are the whole thumbnail. They shrink the same WebP in the browser.

The admin list is admin/posts/index.blade.php line 27, the component tag whose src is $post->displayImageUrl(). Trash repeats that call at trash.blade.php line 17. The img tag is admin-photo-thumb.blade.php line 17. The classes are h-12 w-12 and object-cover. Edit uses the same component at edit.blade.php line 51 with size lg, which is h-24 w-24. The button on line 15 sends that same src into the modal. The modal img is admin-photo-preview-shell.blade.php lines 34 to 37, with object-contain. Still one file.

Do not demo the member page /posts/{id} for the watermark. posts/show.blade.php line 9 calls $post->galleryImageUrl(), not displayImageUrl(). That page shows a packed JPG even when the row has a WebP. The public URL /bouquets/{id} is the page that uses line 5 above.

8. Replace, trash, and delete the file

Edit with no new file leaves the column alone. hasFile('image') is false, so Intervention does not run. Edit with a new file writes the new WebP first, updates the row, then deletes the old path. If the update throws, the new file is deleted and the old path stays. The row never points at a file that was already erased.

Delete on the list calls destroy(), which sets deleted_at. The WebP stays, so Trash can still call displayImageUrl(). Delete in Trash copies the path, removes the row, then Storage::disk('public')->delete() on that products/<uuid>.webp path. The route uses withTrashed() so Laravel can still find a trashed row. Restore does not move a file.

9. What the test proves

php artisan test --filter=LabActFourInterventionTest uses a fake public disk. It posts a JPG and asserts the stored path starts with products/ and ends with .webp. A second test soft-deletes and asserts the file remains. A third force-deletes and asserts the file is gone. The test does not look at watermark pixels. You show the mark with your eyes on /bouquets/{id} or in the admin preview.

10. If sir says make a field required

On Add bouquet these are already required. Name is title. Description is body. Price is price. Category is category_id. Photo is image. The rules are PostRequest.php lines 28 to 32. The matching Blade tags in create.blade.php already have the HTML attribute required. Price also uses a regex of 1 to 16 digits, so a decimal is rejected.

Edit is the field that can change in the room. PostRequest.php line 24 uses nullable for image when the method is PUT. edit.blade.php line 56 is a file input with no required attribute. An empty photo on Edit keeps the old file. If sir says make the photo required, that is the change.

First, in PostRequest.php line 24, replace nullable with required. The line becomes required, image, mimes jpg jpeg png webp, and max 3072. Leave the input name as image. Second, in edit.blade.php line 56, add the attribute required on the file input, the same word create.blade.php line 48 already has. Third, do not add a migration. The image column can stay nullable in the table. The form rule is what sir is asking for.

The message is already written. messages() has image.required set to Please upload a photo for this bouquet. Save both files. Open a bouquet, choose Edit, leave Photo empty, and click Save changes. Laravel returns to the form and shows that sentence under Photo. saveWatermarkedImage() does not run, so the old WebP stays.

If sir points at a field that is already required, say the line and submit the form with that field empty. Do not add required a second time. If sir names a different field, use the same two places. The rule key in PostRequest must match the name attribute on the Blade tag. Put required in the rule array. Put required on the tag. Add a messages() line only when you want your own sentence. The HTML attribute stops a normal browser. PostRequest still checks the request when that attribute is skipped. Show both.

11. What to say if you forget a line

The stamp is public/images/watermark.png. It is a PNG of the words petal express, not letters drawn by Intervention. The other Intervention calls are decode, scaleDown, insert, and encodeUsingFormat to WebP. The saved photo is storage/app/public/products/<uuid>.webp. The display line is Post.php line 62. The public img tag is bouquets-show.blade.php line 5. If sir says make the edit photo required, change nullable to required on PostRequest.php line 24 and add required on edit.blade.php line 56.

There are two picture files. public/images/watermark.png is the shared stamp. The controller always opens that exact name. storage/app/public/products/<uuid>.webp is the bouquet photo. A new UUID keeps a second upload from replacing the first. The visitor's original filename is discarded.

```
Two picture files

Stamp (one shared file, committed with the app)
  petal-express/public/images/watermark.png
  PNG, 560 x 160, transparent background

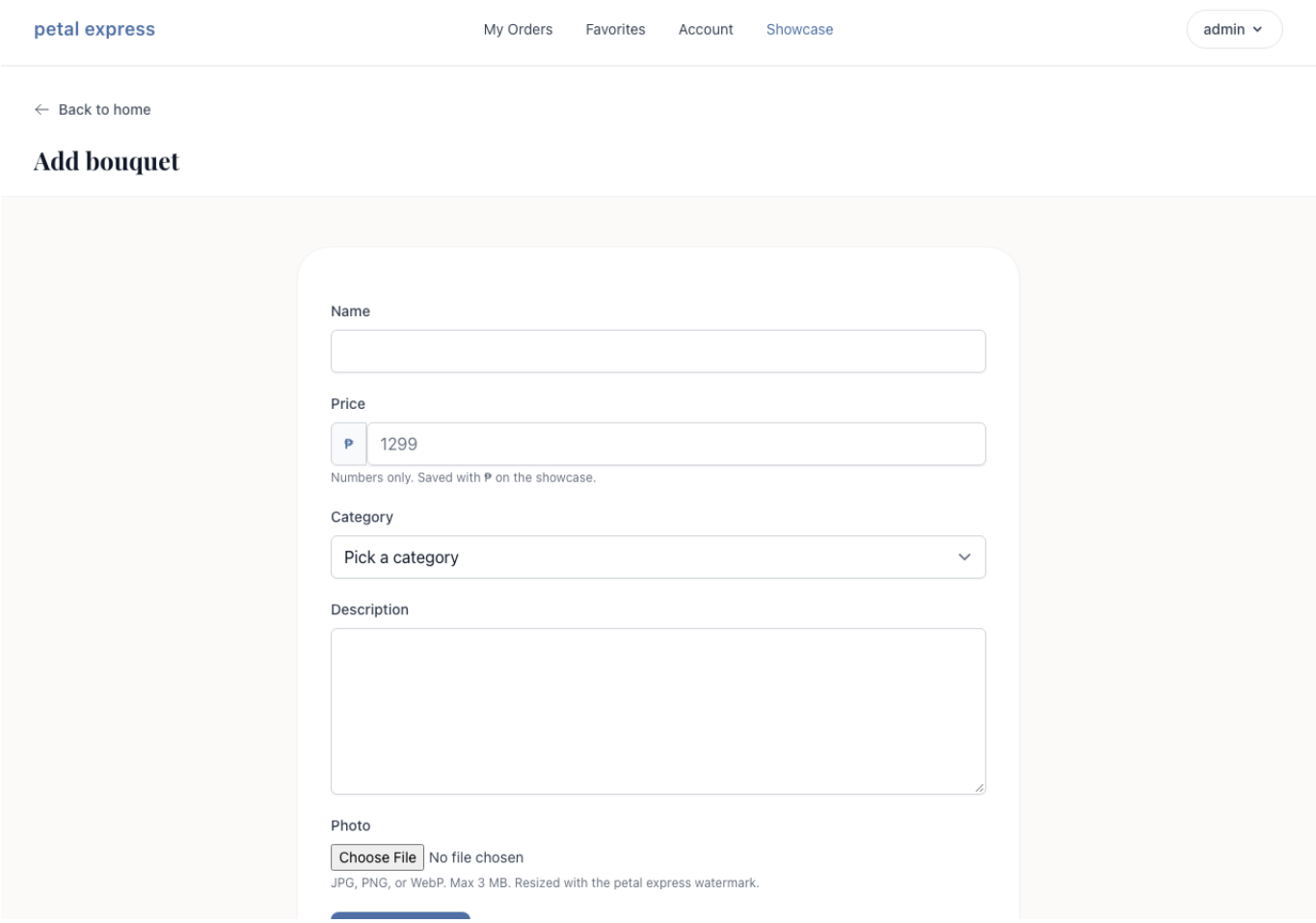
Saved bouquet photo (new file on every upload)
  petal-express/storage/app/public/products/<uuid>.webp
  example: products/8bfef734-0640-407f-af5d-a6f47d24a581.webp
  Pillow says: WebP, 800 x 1150

These are not the same file.
The stamp is not copied into products/.
The upload's original name is not kept.
```

This picture is the stamp file itself, public/images/watermark.png. The pixels are the words petal express. The file is a PNG, 560 by 160, with a transparent background. saveWatermarkedImage() does not draw those letters. It inserts this PNG. A transparent word mark stays readable on flowers. A second floral picture would cover the bouquet. The font and the color already live in the file, so the upload does not depend on a font installed on the laptop.



This is the Add bouquet page in the browser, /admin/posts/create, at 1280 by 900. The Photo field is the new part. JPG, PNG, or WebP, max 3 MB. The rest of the form is the same bouquet card from Labs 2 and 3.



The dark bar is the file name, petal-express/resources/views/admin/posts/create.blade.php. Lines 10 and 11 are the whole form tag. enctype="multipart/form-data" is what puts the photo bytes in the request. A normal form sends text only. @csrf is Laravel's check that the form came from this site.

petal-express/resources/views/admin/posts/create.blade.php

```
10                      <form method="POST" action="{{ route('admin.posts.store') }}"
    enctype="multipart/form-data" class="space-y-5">
11                          @csrf
```

Same file, lines 46 to 49. The input name is image. PostRequest looks for that exact name. accept is only a hint to the file picker. The real check is the PHP rules on the next figure.

petal-express/resources/views/admin/posts/create.blade.php

```
46                          <div>
47                              <x-input-label for="image" value="Photo" />
48                              <input id="image" name="image" type="file"
    accept="image/jpeg,image/png,image/webp" class="mt-2 block w-full text-sm
    text-gray-700" required />
49                              <p class="mt-1 text-xs text-gray-500">JPG, PNG, or WebP.
    Max 3 MB. Resized with the petal express watermark.</p>
```

File petal-express/app/Http/Requests/PostRequest.php, lines 21 to 33. On create, image is required. On edit it is nullable, so a blank file input keeps the old photo. image means PHP recognizes a picture. mimes allows jpg, jpeg, png, and webp. max:3072 is kilobytes, so the cap is 3 MB. A failed rule sends the admin back to the form. The save method never starts.

petal-express/app/Http/Requests/PostRequest.php

```
21      public function rules(): array
22      {
23          $imageRules = $this->isMethod('put') || $this->isMethod('patch')
24              ? ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072']
25              : ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072'];
26
27          return [
28              'title' => ['required', 'string', 'max:255'],
29              'body' => ['required', 'string'],
30              'price' => ['bail', 'required', 'regex:/^\d{1,16}$/'],
31              'category_id' => ['required', 'integer', 'exists:categories,id'],
32              'image' => $imageRules,
33          ];
```

File petal-express/routes/web.php, lines 42 to 46. auth and role:admin run before the controller. A guest goes to login. A member gets 403. Intervention does not run for them. POST /admin/posts is the URL the form uses.

petal-express/routes/web.php

```
42  // admin post pages (must be logged in AND role = admin)
43  Route::middleware(['auth',
    'role:admin'])->prefix('admin')->name('admin.')->group(function () {
44      Route::get('/posts', [AdminPostController::class, 'index'])->name('posts.index');
45      Route::get('/posts/create', [AdminPostController::class,
    'create'])->name('posts.create');
46      Route::post('/posts', [AdminPostController::class, 'store'])->name('posts.store');
```

File petal-express/app/Http/Controllers/AdminPostController.php, lines 35 to 50. store() does not resize the photo. Line 44 calls saveWatermarkedImage and stores the string it returns in the image column. The flash text is Bouquet added. Watermark applied.

petal-express/app/Http/Controllers/AdminPostController.php

```
35      public function store(PostRequest $request): RedirectResponse
36      {
37          $data = $request->validated();
38
39          Post::create([
40              'title' => $data['title'],
41              'body' => $data['body'],
42              'price' => $data['price'],
43              'category_id' => $data['category_id'],
44              'image' => $this->saveWatermarkedImage($request->file('image')),
45              'user_id' => auth()->id(),
46          ]);
47
48          return redirect()
49              ->route('admin.posts.index')
50              ->with('success', 'Bouquet added. Watermark applied.');
```

Same controller, lines 148 to 165. Image::decode reads the upload, so a JPG and a PNG become the same in-memory picture. scaleDown(width: 1000) shrinks only a photo wider than 1000. Height changes by the same factor, so the aspect ratio stays. The stamp path is public/images/watermark.png. For an 800-pixel-wide photo, 38 percent is 304, which is under the 320 cap, so the mark is scaled to 304 wide.

petal-express/app/Http/Controllers/AdminPostController.php

```
148      // resize, watermark, save as webp under storage/app/public/products
149      private function saveWatermarkedImage(UploadedFile $file): string
150      {
151          $image = Image::decode($file);
152          $image->scaleDown(width: 1000);
153
154          $watermarkPath = public_path('images/watermark.png');
155          if (! is_file($watermarkPath)) {
156              throw new \RuntimeException('Watermark not found:
     public/images/watermark.png');
157          }
158
159          $watermark = Image::decode($watermarkPath);
160          // bigger watermark: up to 38% of photo width, capped at 320px
161          $watermarkWidth = min(
162              320,
163              max(1, (int) ($image->width() * 0.38))
164          );
165          $watermark->scaleDown(width: $watermarkWidth);
```

Same file, lines 167 to 185. insert paints the mark 12 pixels in from the bottom-right at 0.55 opacity. The GD driver treats 1 as full strength. The saved name is products/ plus Str::uuid() plus .webp. encodeUsingFormat writes WebP at quality 80. Storage::disk('public')->put writes the bytes. The method returns that path, and that path is all the database stores.

petal-express/app/Http/Controllers/AdminPostController.php

```
167          $image->insert(
168              $watermark,
169              12,
170              12,
171              Alignment::BOTTOM_RIGHT,
172              0.55
173          );
174
175          $path = 'products/'.Str::uuid().'.webp';
176          $encoded = $image->encodeUsingFormat(
177              Format::WEBP,
178              quality: 80
179          );
180
181          if (! Storage::disk('public')->put($path, (string) $encoded)) {
182              throw new \RuntimeException('Failed to save image.');
183          }
184
185          return $path;
```

Read the stored string in three pieces. products/ is the folder. The middle is the UUID from Str::uuid(), shaped 8-4-4-4-12. .webp matches Format::WEBP. The file is written before the new row id exists, so the name is not the id. The browser URL adds /storage/ in front.

Why the saved name looks like that

```
Stored in showcase_flowers.image:
  products/8bfef734-0640-407f-af5d-a6f47d24a581.webp

products/     folder on the public disk
8bfef734-0640-407f-af5d-a6f47d24a581
              Str::uuid()  (8-4-4-4-12 hex)
.webp         Format::WEBP, quality 80

Not used: the visitor filename (rose.jpg)
Not used: the database id
Why: two uploads must not overwrite each other,
and the file is written before the new id exists.

Browser URL:
  /storage/products/8bfef734-0640-407f-af5d-a6f47d24a581.webp
```

File petal-express/config/filesystems.php, lines 41 to 48. The public disk root is storage_path('app/public'), which is petal-express/storage/app/public. The url line would prefix APP_URL. displayImageUrl() skips that helper and returns a path that starts with /storage/.

petal-express/config/filesystems.php

```
41          'public' => [
42              'driver' => 'local',
43              'root' => storage_path('app/public'),
44              'url' => env('APP_URL').'/storage',
45              'visibility' => 'public',
46              'throw' => false,
47              'report' => false,
48          ],
```

File petal-express/database/migrations/2026_09_23_234450_create_showcase_flowers_table.php, lines 14 to 24. The table is showcase_flowers. Line 20 makes image a nullable string. The column holds the path. It does not hold the pixels.

petal-express/database/migrations/2026_09_23_234450_create_showcase_flowers_table.php

```
14          Schema::create('showcase_flowers', function (Blueprint $table) {
15              $table->id();
16              $table->string('title');
17              $table->text('body');
18              $table->string('price')->default('₱0');
19              // optional leftover column; showcase photos cycle from the packed gallery
20              $table->string('image')->nullable();
21              $table->foreignId('user_id')->constrained()->cascadeOnDelete();
22              $table->timestamps();
23              $table->softDeletes();
24          });
```

public/storage is a shortcut to storage/app/public, from php artisan storage:link. Row 101, Progress Report Bouquet, stores products/8bfef734-0640-407f-af5d-a6f47d24a581.webp. The browser asks for /storage/ plus that string.

Symlink and the database path

```
$ ls -ld public/storage
public/storage -> storage/app/public

Disk root:  storage/app/public
Web prefix: /storage
Shortcut:   public/storage   (php artisan storage:link)

sqlite> SELECT id, title, image FROM showcase_flowers
        WHERE id = 101;
101 | Progress Report Bouquet
    | products/8bfef734-0640-407f-af5d-a6f47d24a581.webp

The column stores the path. It does not store pixels.
```

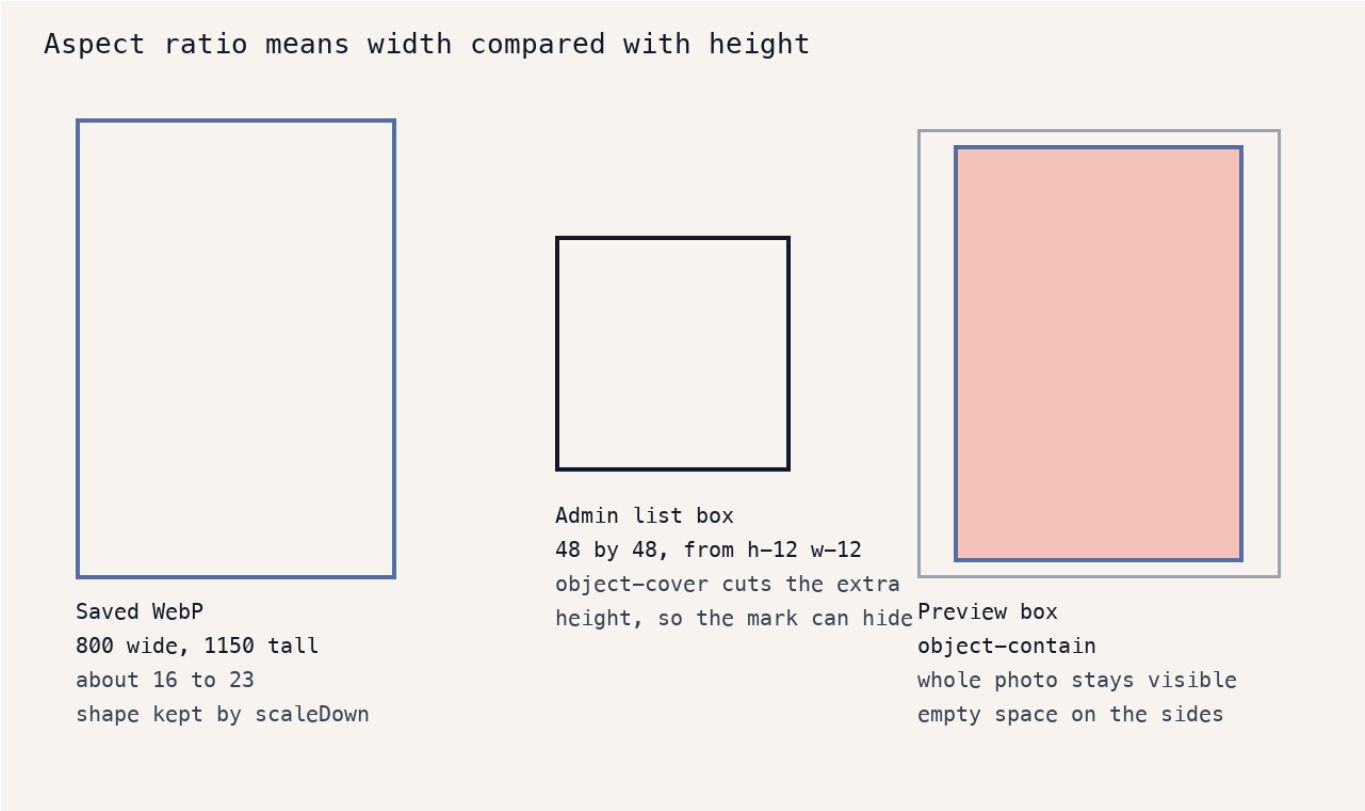
This figure is the saved file itself, storage/app/public/products/8bfef734-0640-407f-af5d-a6f47d24a581.webp. Pillow measures it at 800 by 1150. That ratio is about 16 to 23. Width stayed 800 because it was already under 1000, and height stayed in proportion. The mark is in the bottom-right. This is the only image file Intervention wrote for that upload.



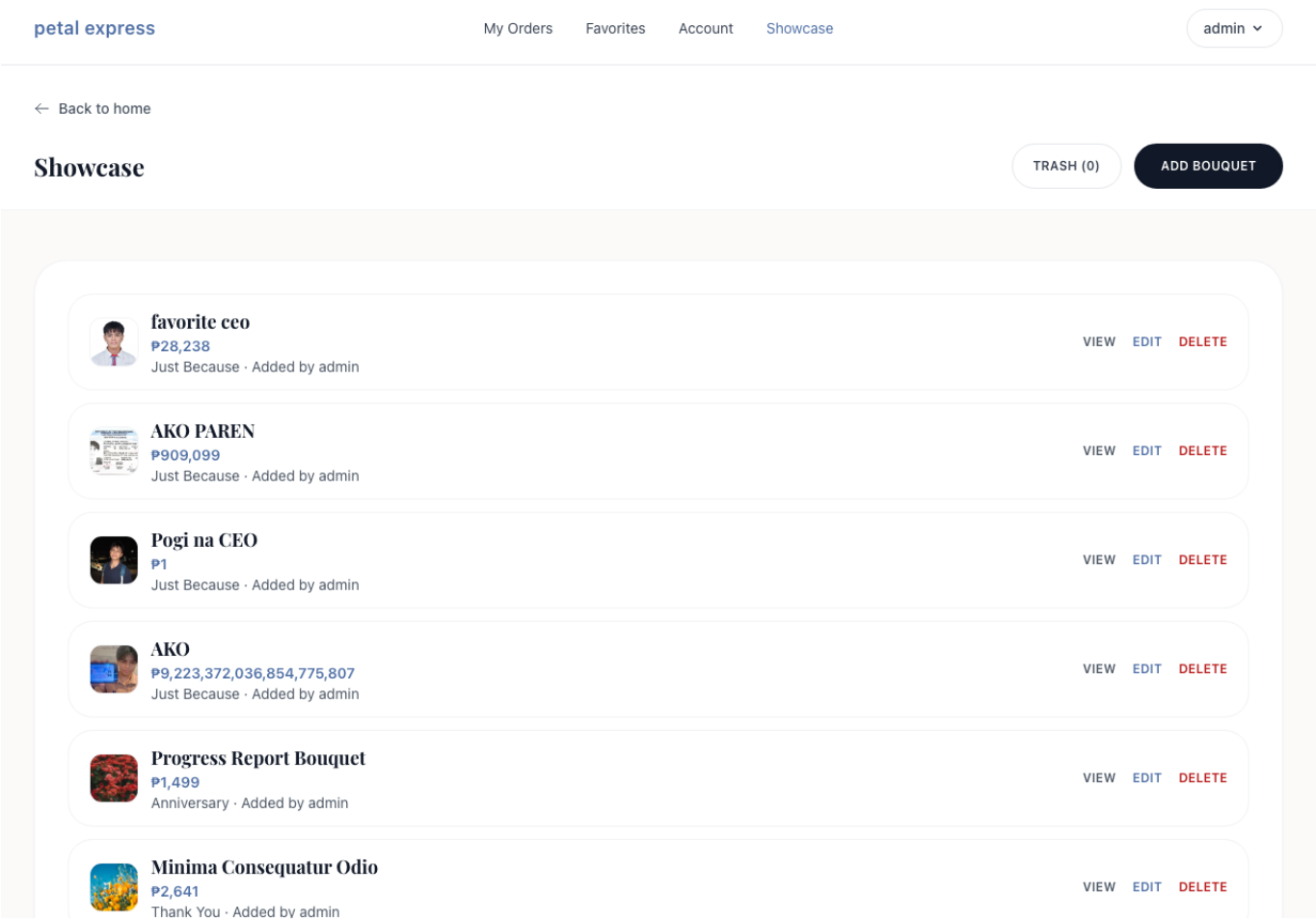
Close-up of the bottom-right of that same WebP. The words are the stamp after scaleDown and insert. The gap from the corner is 12 pixels. 0.55 keeps 55 percent of the mark's strength.



A diagram of the same photo in three boxes. The file stays 800 by 1150. The admin list forces a 48 by 48 square and object-cover cuts the extra height, which can hide the mark. The preview uses object-contain, so the whole tall photo stays visible and empty space appears on the sides.



The admin Showcase page at 1280 by 900. Each row is the Lab 2 and 3 layout, with the watermarked photo on the left and View, Edit, and Delete on the same line. Progress Report Bouquet's small photo is the 800 by 1150 file drawn in a 48 by 48 square. The square shows the middle of the flowers. The mark is in the part object-cover cuts off.



File petal-express/resources/views/admin/posts/index.blade.php, lines 24 to 28. Line 27 is the list photo. :src="$post->displayImageUrl()" is the full WebP URL. The component only decides how big the box is.

petal-express/resources/views/admin/posts/index.blade.php

```
24                          @forelse ($posts as $post)
25                              <div class="rounded-[1.5rem] border border-gray-100 px-4
    py-3 sm:px-5 flex flex-wrap items-center justify-between gap-3">
26                                  <div class="flex items-center gap-3 min-w-0">
27                                      <x-admin-photo-thumb
    :src="$post->displayImageUrl()" :title="$post->title" />
28                                      <div class="min-w-0">
```

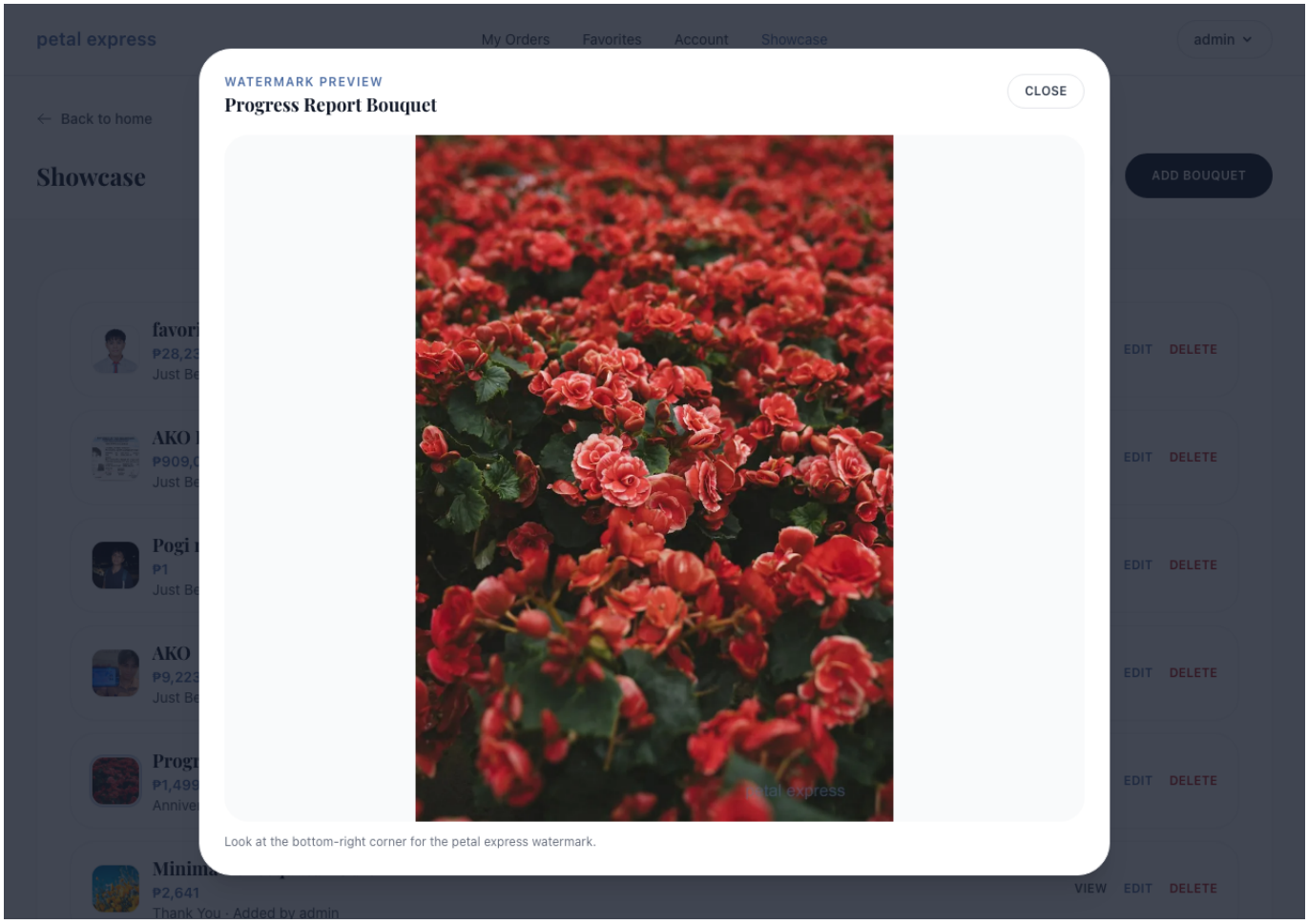
File petal-express/resources/views/components/admin-photo-thumb.blade.php, lines 7 to 17. Line 8 sets the box to h-12 w-12, 48 by 48, unless size is lg. Line 15 sends the same src to the preview. Line 17 is the img tag. object-cover fills the square and crops. There is still one file.

petal-express/resources/views/components/admin-photo-thumb.blade.php

```
@php
    $sizeClass = $size === 'lg' ? 'h-24 w-24' : 'h-12 w-12';
@endphp

<button
    type="button"
    aria-label="Preview photo of {{ $title }}"
    {{ $attributes->merge(['class' => 'shrink-0 rounded-xl focus:outline-none focus:ring-2 focus:ring-accent/30 cursor-zoom-in']) }}
    @click="$dispatch('open-photo-preview', { src: @js($src), title: @js($title) })"
>
    <img src="{{ $src }}" alt="" class="{{ $sizeClass }} rounded-xl object-cover border border-gray-100">
```

Clicking the Progress Report Bouquet photo opens this preview. Playwright measured the img box at 846 by 675 on a 1280 by 900 window. The file inside is still 800 by 1150. object-contain fits the whole shape, so the petal express mark is visible at the bottom-right. Intervention does not run again for this click.

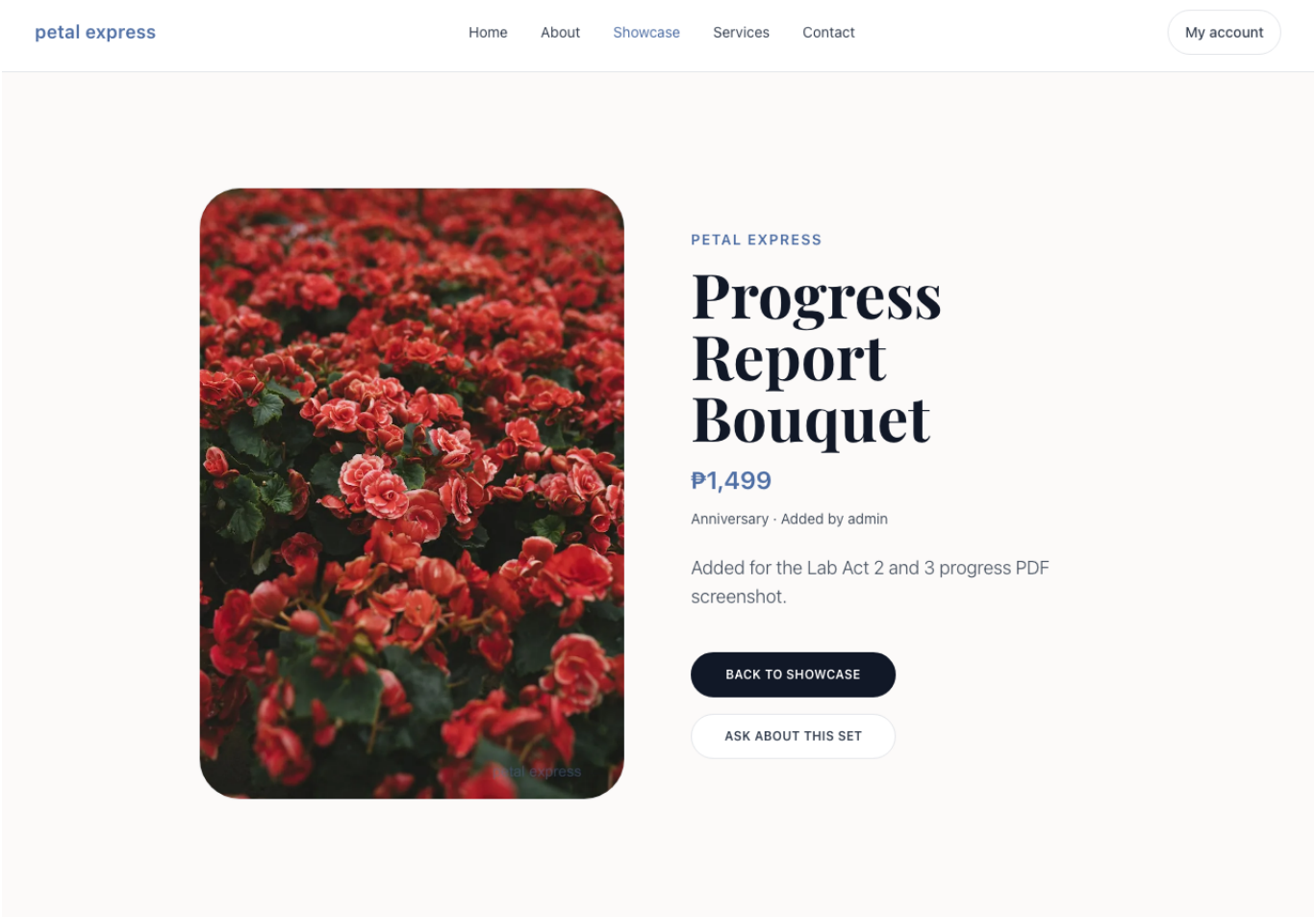


File petal-express/resources/views/components/admin-photo-preview-shell.blade.php, lines 34 to 40. :src="previewSrc" is the URL the thumb already had. object-contain keeps the aspect ratio and letterboxes. max-h-[75vh] stops the photo from growing past 75 percent of the window height.

petal-express/resources/views/components/admin-photo-preview-shell.blade.php

```
34              <img
35                  :src="previewSrc"
36                  alt=""
37                  class="w-full max-h-[75vh] rounded-[1.5rem] object-contain bg-gray-50"
38              >
39
40              <p class="mt-3 text-xs text-gray-500">Look at the bottom-right corner for
    the petal express watermark.</p>
```

The public page /bouquets/101. This is the page to demo. Playwright measured the photo box at 414 by 595. 414 to 595 matches 800 to 1150, so on this window object-cover shows the whole file, mark included. The page only prints an img tag.

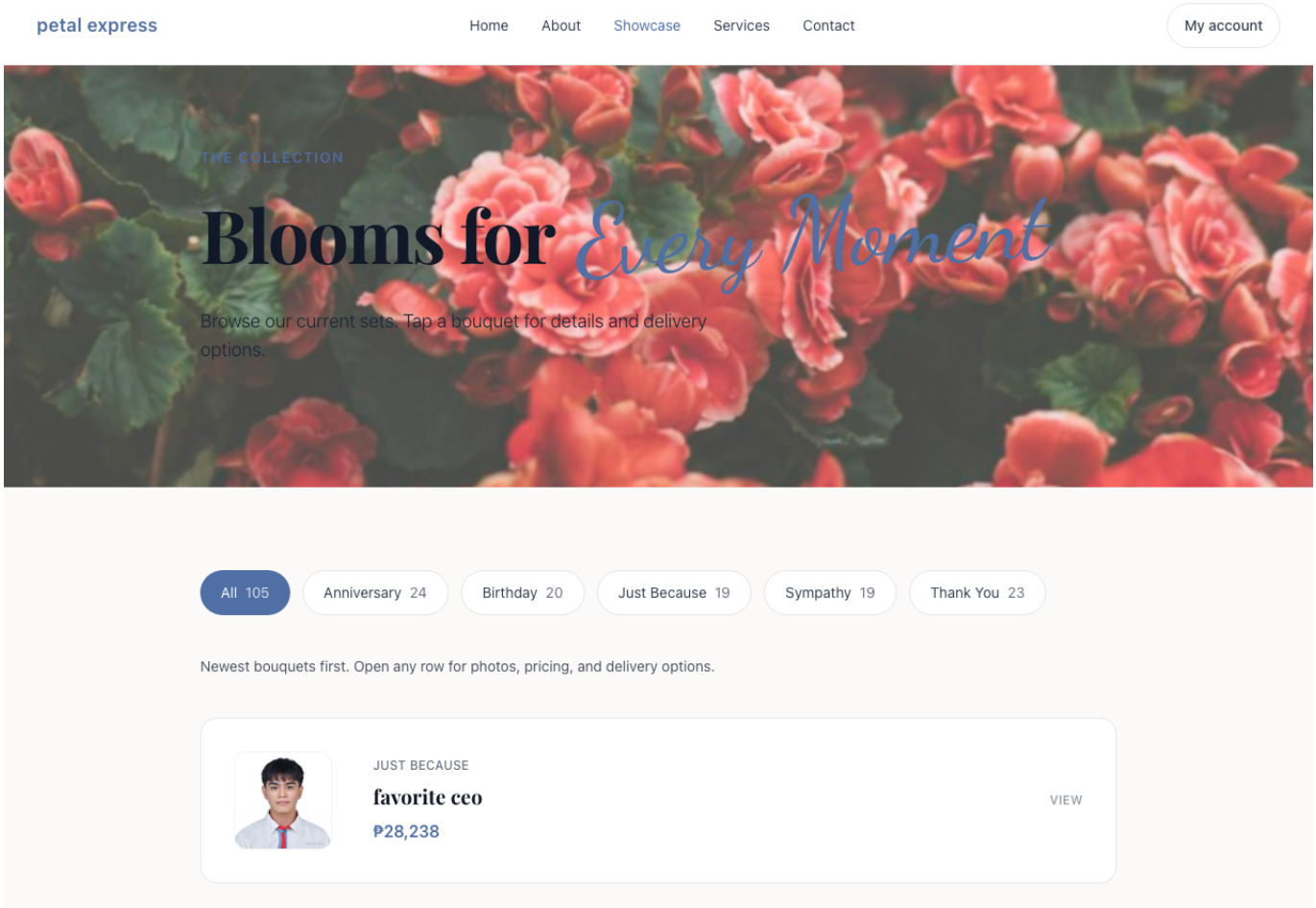


File petal-express/resources/views/pages/bouquets-show.blade.php, lines 4 to 6. Line 5 is the public img tag. displayImageUrl() runs here, when Blade prints the src. object-cover and min-h-[420px] are the box. They do not change the file.

petal-express/resources/views/pages/bouquets-show.blade.php

```
4               <div class="overflow-hidden rounded-[2.5rem] border border-gray-100">
5                   <img src="{{ $bouquet->displayImageUrl() }}" alt="{{ $bouquet->title
    }}" class="h-full w-full object-cover min-h-[420px]">
6               </div>
```

The public showcase list. The small photo on each row is the same WebP URL in a larger CSS box, h-20 w-20, which is 80 by 80, and 96 by 96 from the sm: breakpoint. object-cover is on that box too.



File petal-express/resources/views/pages/bouquets-index.blade.php, lines 44 to 47. Line 47 passes displayImageUrl() into the list component. home.blade.php line 55 and member/favorites.blade.php line 19 do the same call.

petal-express/resources/views/pages/bouquets-index.blade.php

```
44                      <x-showcase-list-item
45                          :href="route('bouquets.show', $bouquet->id)"
46                          :title="$bouquet->title"
47                          :image="$bouquet->displayImageUrl()"
```

File petal-express/resources/views/components/showcase-list-item.blade.php, lines 13 to 16. Line 14 is the img src. The classes h-20 w-20 and object-cover are why the list looks like a thumbnail. The src is still the full WebP.

petal-express/resources/views/components/showcase-list-item.blade.php

```
13          <img
14              src="{{ $image }}"
15              alt="{{ $alt !== '' ? $alt : $title }}"
16              class="h-20 w-20 shrink-0 rounded-xl object-cover border border-gray-100
    sm:h-24 sm:w-24"
```

File petal-express/app/Models/Post.php, lines 56 to 78. Line 62 is the line that displays an upload. It returns /storage/ plus the column. galleryImageUrl() on line 70 runs only when the column is empty or the file is missing. It picks public/images/bouquets/01.jpg through 06.jpg. Those JPGs have no watermark.

petal-express/app/Models/Post.php

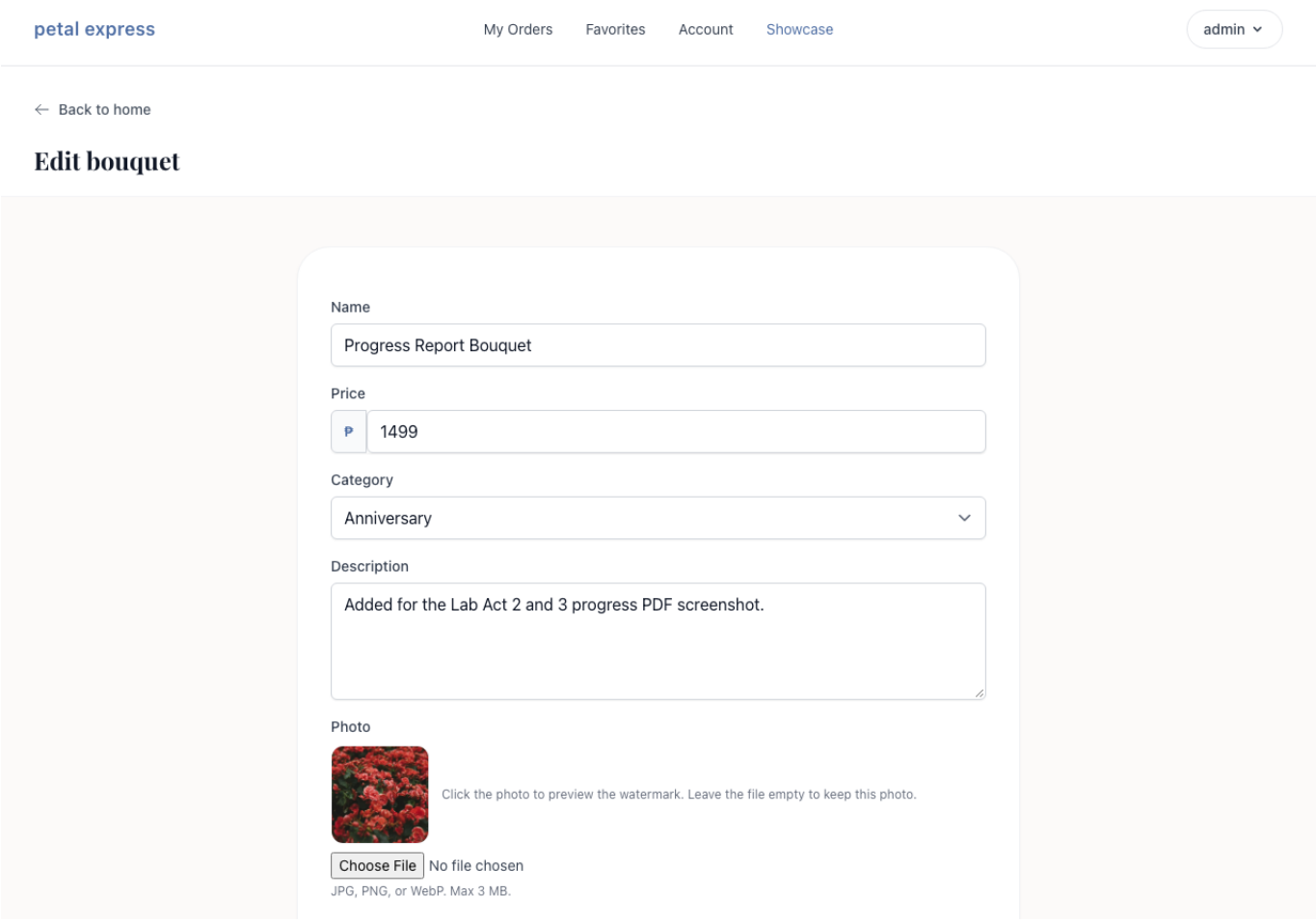
```
56      // uploaded webp in storage, or the packed gallery when image is empty
57      public function displayImageUrl(): string
58      {
59          if ($this->image) {
60              // relative path so images work on 127.0.0.1:8000 even when APP_URL is
    http://localhost
61              if (Storage::disk('public')->exists($this->image)) {
62                  return '/storage/'.ltrim($this->image, '/');
63              }
64          }
65
66          return $this->galleryImageUrl();
67      }
68
69      // pick one of the 6 packed images by id (cycles when id > 6)
70      public function galleryImageUrl(): string
71      {
72          $count = count(self::GALLERY);
73          $index = ((int) $this->id - 1) % $count;
74          if ($index < 0) {
75              $index = 0;
76          }
77
78          return asset(self::GALLERY[$index]);
```

File petal-express/resources/views/posts/show.blade.php, lines 8 and 9. Line 9 calls galleryImageUrl(), so /posts/{id} shows a packed JPG even when the row has a WebP. Demo the watermark on /bouquets/{id}.

petal-express/resources/views/posts/show.blade.php

```
8               <div class="overflow-hidden rounded-2xl border border-gray-200 bg-white">
9                   <img src="{{ $post->galleryImageUrl() }}" alt="{{ $post->title }}"
    class="h-72 w-full object-cover sm:h-80">
```

The edit page for bouquet 101. The photo is the same WebP at size lg, 96 by 96. Leave the file input empty and the old path stays. Choose a new file and the controller writes the new WebP before it deletes the old one.



Intervention writes one file. There is no products/<uuid>-thumb.webp. The list, the edit page, the preview, and /bouquets/101 all request that one URL. CSS decides the box. Delete forever removes that one path.

The thumbnail is the same WebP drawn small

```
Intervention writes ONE file:
  storage/app/public/products/8bfef734-0640-407f-af5d-a6f47d24a581.webp

There is no products/<uuid>-thumb.webp.

Admin list box:  h-12 w-12, 48 by 48, object-cover
Preview modal:   same URL, object-contain
Public detail:   same URL, object-cover, min-h 420px

Method they all call:
  Post::displayImageUrl()
  return '/storage/' + image column
```

File petal-express/app/Http/Controllers/AdminPostController.php, lines 97 to 104. destroy() calls delete(), which only sets deleted_at. The WebP stays in storage/app/public/products, so Trash can still show it.

petal-express/app/Http/Controllers/AdminPostController.php

```
97      // sets deleted_at. the row stays in the table (image file stays too)
98      public function destroy(Post $post): RedirectResponse
99      {
100         $post->delete();
101
102         return redirect()
103             ->route('admin.posts.index')
104             ->with('success', 'Bouquet moved to Trash.');
```

Same file, lines 127 to 141. forceDelete copies the path, removes the row, then Storage::disk('public')->delete on products/<uuid>.webp. The route uses withTrashed() so Laravel can still load a row that is already in Trash. Restore does not move a file, because the file never left.

petal-express/app/Http/Controllers/AdminPostController.php

```
127      public function forceDelete(Post $post): RedirectResponse
128      {
129          if (! $post->trashed()) {
130              return redirect()
131                  ->route('admin.posts.index')
132                  ->with('error', 'Move the bouquet to Trash before deleting it.');
133          }
134
135          $imagePath = $post->image;
136
137          $post->forceDelete();
138
139          if ($imagePath) {
140              Storage::disk('public')->delete($imagePath);
141          }
```

File petal-express/app/Http/Controllers/AdminPostController.php, lines 148 to 165. Intervention is used for more than the stamp. Image::decode reads a JPG, a PNG, or a WebP into one picture. scaleDown shrinks the photo when it is wider than 1000 pixels and keeps the aspect ratio. The same scaleDown then sizes watermark.png. There is no text() call and no blur, crop, or flip.

petal-express/app/Http/Controllers/AdminPostController.php

```
148      // resize, watermark, save as webp under storage/app/public/products
149      private function saveWatermarkedImage(UploadedFile $file): string
150      {
151          $image = Image::decode($file);
152          $image->scaleDown(width: 1000);
153
154          $watermarkPath = public_path('images/watermark.png');
155          if (! is_file($watermarkPath)) {
156              throw new \RuntimeException('Watermark not found:
     public/images/watermark.png');
157          }
158
159          $watermark = Image::decode($watermarkPath);
160          // bigger watermark: up to 38% of photo width, capped at 320px
161          $watermarkWidth = min(
162              320,
163              max(1, (int) ($image->width() * 0.38))
164          );
165          $watermark->scaleDown(width: $watermarkWidth);
```

File petal-express/app/Http/Requests/PostRequest.php, lines 21 to 33. On Add bouquet, title, body, price, category, and image are already required. On Edit, line 24 says nullable for the photo, so an empty file input is allowed. If sir says make the edit photo required, change nullable to required on that line. The input name in Blade must stay image, because that is the key in this array.

petal-express/app/Http/Requests/PostRequest.php

```
21      public function rules(): array
22      {
23          $imageRules = $this->isMethod('put') || $this->isMethod('patch')
24              ? ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072']
25              : ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072'];
26
27          return [
28              'title' => ['required', 'string', 'max:255'],
29              'body' => ['required', 'string'],
30              'price' => ['bail', 'required', 'regex:/^\d{1,16}$/'],
31              'category_id' => ['required', 'integer', 'exists:categories,id'],
32              'image' => $imageRules,
33          ];
```

File petal-express/resources/views/admin/posts/edit.blade.php, lines 48 to 57. Line 56 is the file input. It has no required attribute. create.blade.php line 48 already has required. For the live change, add required to this edit input as well. The browser then blocks an empty photo. PostRequest is still the check that matters if the browser check is skipped.

petal-express/resources/views/admin/posts/edit.blade.php

```
48                             <div>
49                                 <x-input-label for="image" value="Photo" />
50                                 @if ($post->image)
51                                     <div class="mt-2 flex items-center gap-3">
52                                         <x-admin-photo-thumb
    :src="$post->displayImageUrl()" :title="$post->title" size="lg" />
53                                         <p class="text-xs text-gray-500">Click the
    photo to preview the watermark. Leave the file empty to keep this photo.</p>
54                                     </div>
55                                 @endif
56                                 <input id="image" name="image" type="file"
    accept="image/jpeg,image/png,image/webp" class="mt-2 block w-full text-sm
    text-gray-700">
57                                 <p class="mt-1 text-xs text-gray-500">JPG, PNG, or
    WebP. Max 3 MB.</p>
```

File petal-express/app/Http/Requests/PostRequest.php, lines 68 to 79. image.required already says Please upload a photo for this bouquet. After you change nullable to required, submit Edit with no new file. That sentence appears under Photo, and saveWatermarkedImage() does not run. You do not need a migration. required here is a form rule, not a new column.

petal-express/app/Http/Requests/PostRequest.php

```
68          return [
69              'title.required' => 'Please give the bouquet a name.',
70              'body.required' => 'Please add a short description.',
71              'price.required' => 'Please add a price.',
72              'price.regex' => 'Use up to 16 digits, with no decimals.',
73              'category_id.required' => 'Please pick a category.',
74              'category_id.exists' => 'That category is not in the list.',
75              'image.required' => 'Please upload a photo for this bouquet.',
76              'image.image' => 'The file must be a photo.',
77              'image.mimes' => 'Use JPG, PNG, or WebP.',
78              'image.max' => 'The photo must be 3 MB or smaller.',
79          ];
```